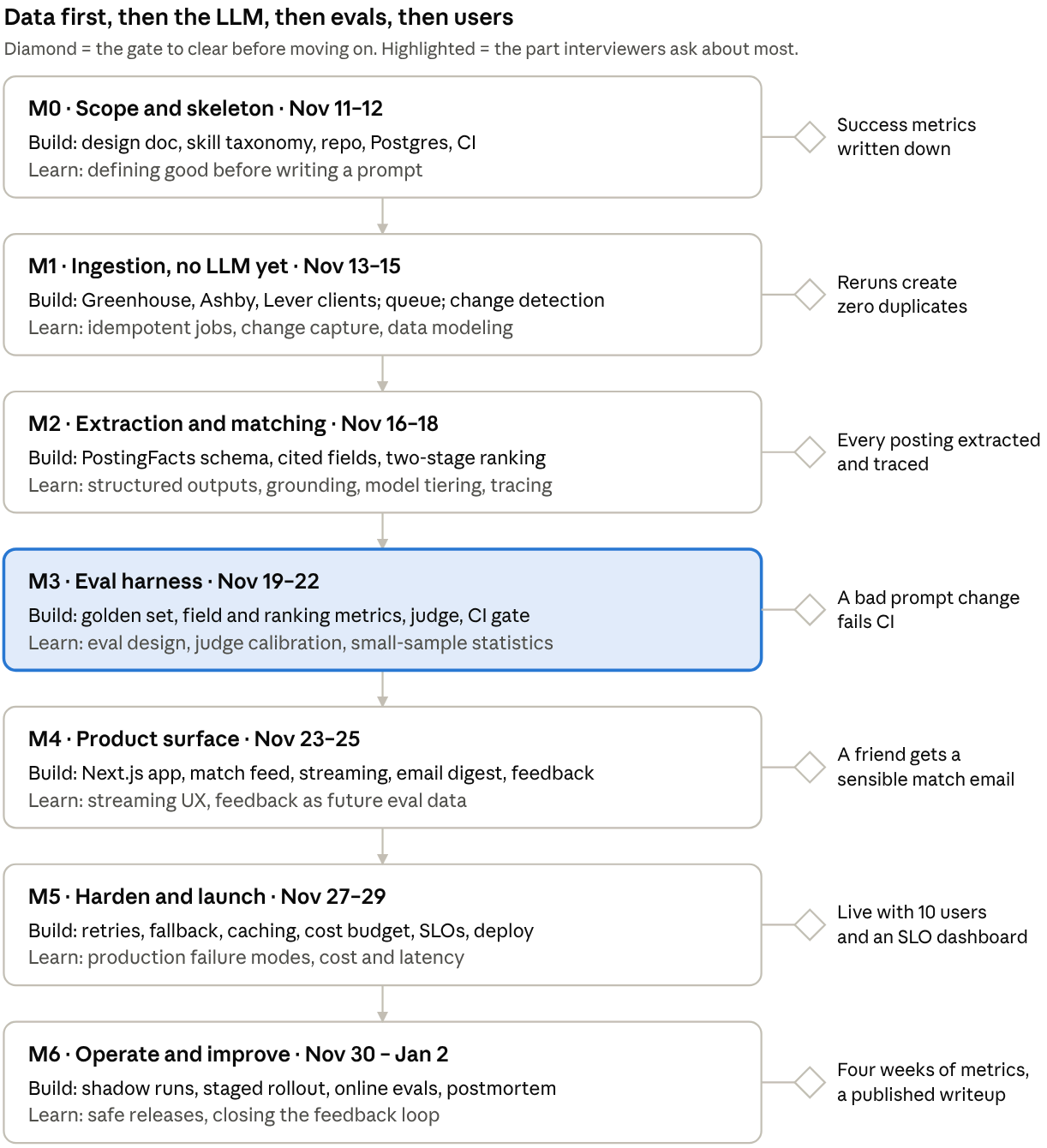
AI Job Radar Build Plan

Nov 11, 2026 · @Nishi

AI Job Radar watches AI startup job boards, turns each posting into structured fields (skills, pay, seniority, remote policy), and emails people the roles that match them. It is built in seven milestones over three weeks (data first, then the LLM, then evals, then users), then run in production through week 8. You are its first user.

Roadmap

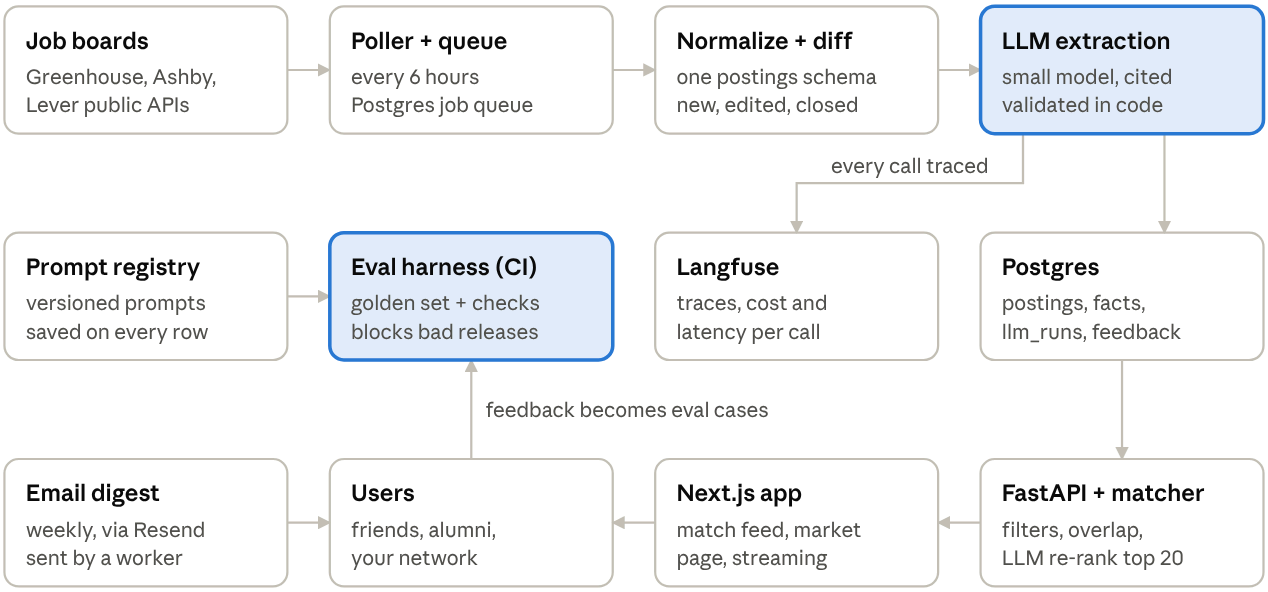


build roadmap · 7 milestones, a gate after each

About 42 hours over the first three weeks at 12–15 hours a week, then about 2 hours a week of running it while you build Projects 2 and 3.

Architecture

Every LLM output is versioned, traced and evaluated before users see it



architecture · pipeline, quality loop and app

The top row runs without any users. The middle row is the quality loop, and the bottom row is what users see. User feedback flows back into the eval set.

Stack

Python for the pipeline and evals, TypeScript for the app. Both languages show up in the postings, and you already know both.

Layer	Choice	Why
Pipeline and API	Python 3.12, httpx, FastAPI, Pydantic	Pydantic models double as structured-output schemas
Job queue	Postgres with `SELECT … FOR UPDATE SKIP LOCKED`	One less service, and you learn how queues actually work
Database	Postgres (Neon), pgvector only if needed	You know it; Neon has a free tier

Layer	Choice	Why
LLM calls	Anthropic and OpenAI SDKs directly, no framework	Learn the raw APIs first; two providers make fallback real
Tracing	OpenTelemetry into Langfuse	Vendor-neutral; Langfuse is named in FieldGuide's posting
Evals	pytest-style harness you write, results in Postgres	You own the metrics; Braintrust or LangSmith can come later
Web app	Next.js, TypeScript, Vercel AI SDK	The stack product-leaning roles ask for
Email	Resend	Simple transactional email
Hosting	API and workers on Railway or Fly.io, web on Vercel	Cheap, quick to deploy, and has staging environments

M0 — Scope and skeleton (Nov 11–12, about 4 hours)

Decide what "good" means before you write a single prompt.

Build

- [ ] One-page design doc: who the user is, the alert they get, which fields you extract, what's out of scope
- [ ] Success metrics with your own targets: field accuracy (salary, seniority, track), skill precision and recall, match quality, p95 latency, cost per posting
- [ ] A fixed skill taxonomy, starting from the 34 skill codes in your 50-posting analysis
- [ ] Repo with `pipeline/` (Python), `web/` (Next.js), `evals/`, `infra/`; Docker Compose with Postgres; Alembic migrations
- [ ] GitHub Actions running lint, type checks and tests
- [ ] Seed list of about 150 companies: the 50 from your analysis plus the Fast AI Jobs Series A–C lists, each tagged with its ATS (Greenhouse, Ashby or Lever), board name and funding stage

What you're learning

- Scoping an LLM feature: deciding what the model does and what plain code does
- Writing success criteria first, so the evals in M3 have something to measure

- Designing a fixed output vocabulary (the taxonomy) so outputs can be scored

Done when: the design doc is committed, `docker compose up` runs, and CI is green.

M1 — Ingestion pipeline, no LLM yet (Nov 13–15, about 8 hours)

Get a clean, stable stream of postings into Postgres. Most LLM product quality comes from this layer.

Build

- [ ] ATS clients for the public, keyless job-board APIs: Greenhouse (`boards-api.greenhouse.io/v1/boards/{token}/jobs?content=true`), Ashby (`api.ashbyhq.com/posting-api/job-board/{name}?includeCompensation=true`) and Lever
- [ ] Poller on a schedule (every 6 hours is plenty) that enqueues one job per board; safe to run twice
- [ ] Postgres job queue with attempts, exponential backoff and a dead-letter status
- [ ] Normalizer: one `postings` schema across all three ATSs (company, title, location, departments, raw HTML, cleaned text, ATS compensation fields, first seen, last seen)
- [ ] Change detection: new, edited (content hash changed) and closed (missing from the board) postings
- [ ] Split each description into numbered paragraphs, stored with IDs, so the LLM can cite them in M2
- [ ] Polite client: a descriptive User-Agent, low concurrency, caching of raw responses
- [ ] Two weeks of history for the seed list before moving on

What you're learning

- Idempotency: jobs that can crash and rerun without creating duplicates
- Change data capture: noticing edits and closures, not just new rows
- Data modeling for LLM inputs: clean text and stable paragraph IDs make every later step easier
- Free labels: Ashby can return structured compensation when the company publishes it, which gives you ground truth for salary extraction in M3
- Data etiquette: the content belongs to the employer, so link back to the original posting, don't republish full descriptions, and keep the project non-commercial

Done when: every board ingests cleanly; rerunning the poller creates zero duplicates; you can query "new Applied AI postings this week."

M2 — LLM extraction and matching (Nov 16–18, about 8 hours)

Turn each posting into a structured record you can score, and rank postings for a person.

Build: extraction

- [ ] Pydantic schema `PostingFacts`: role track (platform, product, forward-deployed, research, other), seniority, minimum years, salary min/max/currency (only if stated), location, remote policy, visa sponsorship, required skills and preferred skills (from the taxonomy only), and an evidence paragraph ID for every field
- [ ] Extraction with a small, cheap model through raw provider SDKs, using structured outputs or tool calling for JSON
- [ ] Validation in code: schema check, skills must be in the taxonomy, evidence IDs must exist, salary numbers must appear in the cited paragraph; one repair retry, then mark the posting as failed
- [ ] Prompt registry: prompts in versioned files; the version is saved on every output row
- [ ] `llm_runs` table: model, prompt version, input and output tokens, cost, latency, outcome
- [ ] OpenTelemetry tracing into Langfuse for every call

Build: matching

- [ ] User profile from a pasted resume, extracted into the same skill taxonomy, which the user can then edit
- [ ] Two-stage ranking: hard filters in code (location, seniority, track), then a skill-overlap score, then a stronger model re-ranks the top 20 and writes a short "why it matches" plus "skills you'd need" note, citing the posting's paragraphs

What you're learning

- Structured outputs and schema design: what to ask the model and what to compute in code
- Grounding by reference: cite paragraph IDs that code can check, never free-text quotes
- Model tiering: a cheap model for high-volume extraction, a stronger one for the few re-ranked results

- Cost accounting from day one, and reading a trace to debug one bad output

Done when: every ingested posting has a validated `PostingFacts` record and a trace, you know the cost per posting, and your own match feed looks sensible.

M3 — Eval harness (Nov 19–22, about 9 hours)

Make quality a number that can block a release. This is the milestone interviewers will ask about most.

Build: extraction evals

- [ ] Golden set of about 200 postings, split by track, stage and ATS, labeled by you with a small Streamlit labeling tool; keep 20% as an untouched holdout
- [ ] Automatic labels: wherever the ATS publishes structured pay, location or remote status, use it as ground truth for those fields
- [ ] Field-level metrics: exact match for seniority, track and remote policy; salary within tolerance; precision and recall for required and preferred skills
- [ ] Code checks on every output: schema valid, evidence IDs exist, salary numbers appear in the cited text, no skills outside the taxonomy

Build: matching evals

- [ ] Relevance labels: rate the top 20 results for 5–10 real profiles (yours plus friends'), then compute NDCG@10 and precision@5
- [ ] LLM-as-judge for whether each "why it matches" note is faithful to the posting; hand-label 50 notes first and report the judge's agreement with you (for example Cohen's kappa) before trusting it

Build: the release gate

- [ ] Eval report with bootstrap confidence intervals, so you can tell noise from a real improvement
- [ ] GitHub Action that runs the suite whenever prompts or model config change, and fails on a regression past thresholds you set
- [ ] Error-analysis notebook: read 30 failures, group them by type, fix the biggest group, re-run

What you're learning

- Eval design: datasets that look like real traffic, holdouts, and not overfitting prompts to your own test set

- Choosing metrics that match the product (does a missed match cost more than a bad one?)
- Ranking evals (NDCG, precision@k), the same tools search and recommendation teams use
- Judge calibration: an unchecked judge is just one more model you haven't measured
- Small-sample statistics: when a 2-point gain is just noise
- Error analysis, which is the real improvement loop: look at the data, sort failures into types, fix, measure

Done when: a deliberately bad prompt change fails CI, and you have one before-and-after improvement backed by numbers.

M4 — Product surface (Nov 23–25, about 6 hours)

Put it in front of people, and capture every reaction as future eval data.

Build

- [ ] FastAPI endpoints: profile, match feed, posting detail, feedback
- [ ] Next.js app with magic-link sign-in: paste a resume, review the extracted skills, set filters (track, location, remote, minimum pay)
- [ ] Match feed: each card shows the extracted fields, the "why it matches" note, and a link to the original posting
- [ ] Posting page: extracted fields with their evidence paragraphs highlighted
- [ ] "What would I need to learn for this role?" streamed with the Vercel AI SDK, cached after the first generation
- [ ] Market page: skill frequency across all tracked postings, refreshed weekly (a live version of your 50-posting analysis)
- [ ] Weekly email digest of new matches through Resend, sent by a worker job
- [ ] Feedback on every card and email (good match, not relevant, applied, wrong field), stored with the prompt version that produced it

What you're learning

- Streaming UX: when it helps (long explanations) and when it doesn't (extraction)
- Designing feedback so it can become labeled eval data later
- One schema across the stack: the same shape drives the Pydantic models, the API and the UI

Done when: a friend signs up, pastes a resume and gets a match email with roles that make sense to them.

M5 — Harden and launch (Nov 27–29, about 7 hours)

Make it survive real traffic and real failures, then launch. Thanksgiving (Nov 26) is a day off.

Build

- [ ] Retries with backoff and jitter, timeouts, and a circuit breaker for each provider
- [ ] Provider fallback (Anthropic and OpenAI); the fallback model must also pass the eval suite
- [ ] Graceful degradation: if extraction fails, show the posting with its raw ATS fields and a "details pending" label instead of hiding it
- [ ] Prompt caching on the long shared instructions; measure the savings alongside model tiering
- [ ] Rate-limit handling and a daily cost budget that alerts you
- [ ] SLOs with your own targets: new posting reaches matched users within N hours, extraction failure rate, p95 page load; a dashboard for each
- [ ] Runbook and incident log template
- [ ] Deploy: API and workers on Railway or Fly.io, web on Vercel, Postgres on Neon; secrets management; a staging environment
- [ ] Recruit 10–20 users: job-seeking friends, Cornell alumni, your network; onboarding message

What you're learning

- How LLM features fail in production: timeouts, 429s, 5xx errors, truncated or malformed output, and quality slowly drifting with nothing technically broken
- Cost and latency engineering with real numbers
- SLOs for LLM features: what to promise users and what to alert yourself on

Done when: it's live with at least 10 users, an SLO dashboard, and a first weekly cost report.

M6 — Operate and improve (Nov 30 – Jan 2, about 2 hours a week)

Run it like a real product while you build Projects 2 and 3. This is what turns a demo into a production story.

Build

- [ ] Shadow runs: a new prompt or model version processes live postings in the background, and you compare it with the current one before switching
- [ ] Staged rollout: promote the winner to a slice of new postings first, then all of them
- [ ] Online evals: a nightly job samples recent outputs, runs the code checks and the judge, and posts a report
- [ ] Drift watch: flag skills and tool names the taxonomy doesn't cover yet (new frameworks show up often), then update the taxonomy on purpose
- [ ] Weekly feedback triage: review every "wrong field" and "not relevant," and add the good cases to the golden set
- [ ] One written postmortem for a real incident (there will be one)
- [ ] Writeup: README with architecture, metrics and eval results; a 3-minute demo video; a blog post on how evals run your release process

What you're learning

- Releasing non-deterministic systems safely: shadow runs, staged rollouts, rollback
- Closing the loop: production, then feedback, then evals, then better prompts
- Telling the story with numbers, which is what your interviews will run on

Done when: you have four weeks of production metrics, one release your evals blocked, one improvement shipped with numbers, and a published writeup.

Sources

- Greenhouse Job Board API documentation
- Ashby public job posting API documentation
- Fastcrw: Ashby job boards provider page (licence and attribution notes)